NEXTGEN QA

NextGen QA

An AI-powered system that transforms Agile conversations into comprehensive, self-healing test suites



Project Supervision

Supervisor



Ms.Suriyaa Kumari

Senior Lecturer

Co-Supervisor



Mr.Eishan Weerasinghe

Lecturer

NEXTGEN QA - VOICE DRIVEN TEST CASE GENERATION AND SELF-HEALING

Overall Project • Comments for the Proposal and how you/team addressed them

Comment Received on Proposal

PANEL / SUPERVISOR FEEDBACK

"Generating Selenium, Playwright, and Cypress code from the same Gherkin is useful engineering, but it is not strong enough as a research novelty — multiple existing tools already do this. The dissertation needs a clearer, defendable research contribution."

"Provide option for Agile Meetings to connect with Slack "

How We Addressed It

BEFORE • ORIGINAL PROPOSAL

Multi-Framework Code Generation

Limitation: Engineering value, but no strong research contribution.

AFTER • STRENGTHENED NOVELTY

Autonomous Agent Explorer (Stage ★)

One-sentence intent triggers a vision-grounded autonomous agent that explores the live application and discovers test scenarios the QA never wrote.

BEFORE • ORIGINAL PROPOSAL

Agile meetings are platform-only.

Limitation: Agile meetings can only be joined through our provided system.

AFTER • Improvement

Provides plugin integration with Slack.

Help easy remote team coordination and Channel-based collaboration

NEXTGEN QA - VOICE DRIVEN TEST CASE GENERATION AND SELF-HEALING

Overall Project • Problem, Beneficiaries & Design Excellence

Problem & Its Impact / Gravity

THE PROBLEM

Manual software testing is slow, fragile, and unable to keep with modern agile and DevOps release cycles.

30%+ of software dev cost goes to QA & testing

70% of test scripts break with every UI change

Days of manual regression bottleneck CI/CD

High compliance risk from manual RTM in regulated sectors

Who Has This Problem / Who Benefits

QA — **QA Engineers & Test Automation Teams**

Direct primary users — productivity multiplier

ORG — **Software Development Organisations**

SMEs to enterprises — lower QA cost, faster releases

DEV — **DevOps Teams**

Unblock CI/CD pipelines with continuous AI testing

REG — **Regulated Industries (Finance / Health)**

Automated RTM for audit and compliance readiness

Design Excellence of the Overall Solution

An end-to-end AI-powered QA lifecycle - four research-backed components working as one integrated pipeline:

C1

User Story Generation

Speech-to-text + LLM extracts user stories from agile meeting recordings.

C2

Test Case Generation

Multi-stage pipeline + autonomous Set-of-Mark agent (research novelty).

C3

Failure Analysis

ML pattern matching auto-repairs broken locators & assertions.

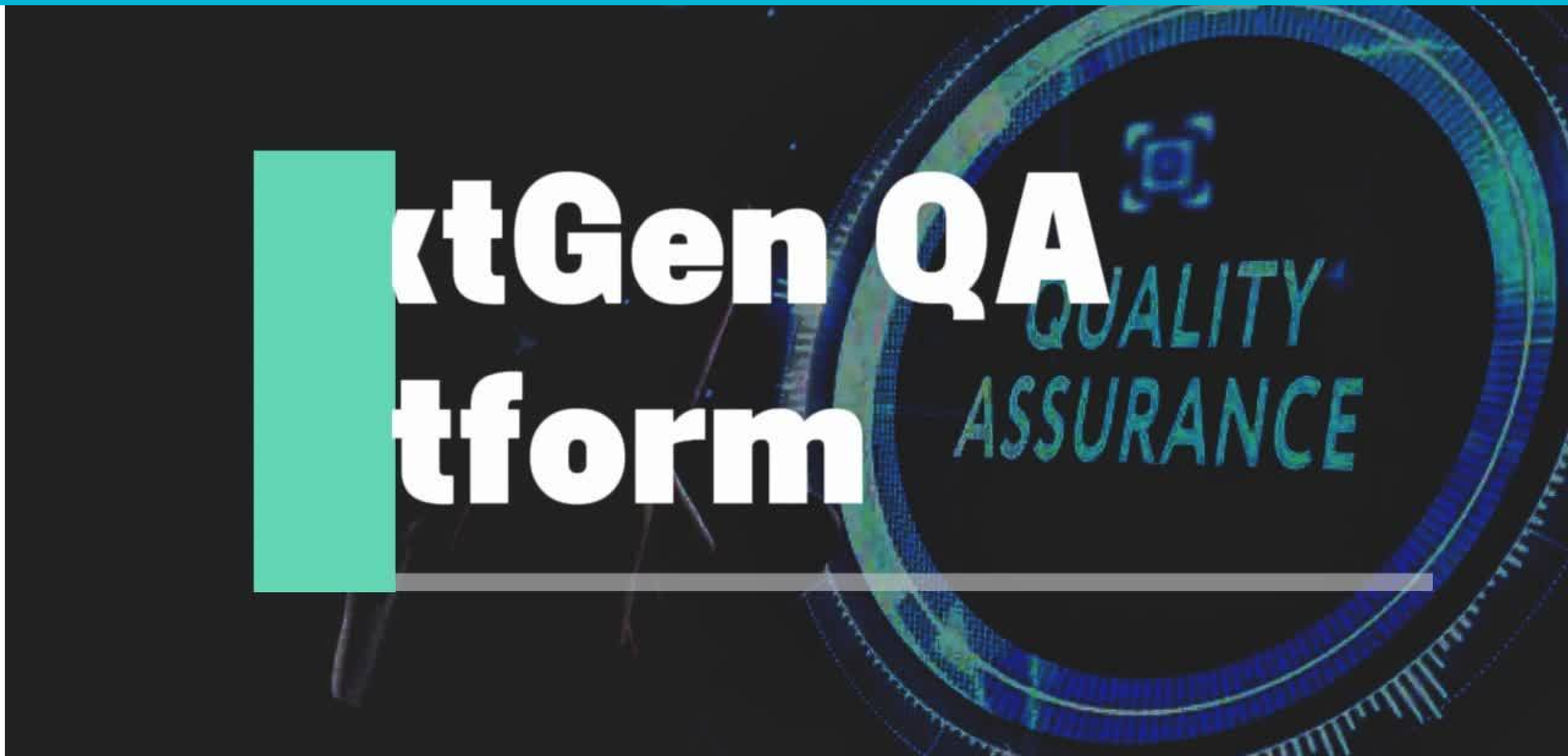
C4

Quality Prediction

Risk scoring + automatic requirement-to-test traceability matrix.

NEXTGEN QA - VOICE DRIVEN TEST CASE GENERATION AND SELF-HEALING

Overall Project



IT22251114 | Jinad Induvidwa A.G



AI driven User story Generation And Conflict Detector

Specialization: Software Engineering

AI driven User story Generation And Conflict Detector

Individual Component • 50% Complete

The Sub-Problem

- Manual requirement capture, some information can be lost.
- Existing tools convert speech to text Only
- Current systems not automatically detect conflicting requirements

Our solution

- Identifies speakers and transcribes meetings in real time.
- Converts messy live speech into structured user stories.
- Detects conflicts in requirements & instantly shows as Warning

Outcome

- Improved requirement accuracy through conflict detection
- Automatic conversion of unstructured speech into user stories.

User Requirements Addressed

✓ Convert voice into real-time transcription.	Done
✓ Save transcriptions into the database for future use.	Done
✓ Convert transcriptions into user stories	Done
✓ Option to manually upload transcript files and generate user stories.	Done
◐ Real time requirement conflicts detector	In progress

Conflict Detection: *No conflict detect yet*

AI driven User story Generation And Conflict Detector

Individual Component • 50% Complete

Design Excellence / Contribution

1. Built a scalable microservices system
2. Integrated secure authentication and cloud-based database (Supabase with pgSql)
3. Implemented real-time voice transcription using WebSocket streaming.
4. Created an RAG for converts meeting transcripts into structured user stories
5. Developed a modern React-based UI with reusable components

User Feedback on Prototype

Collected from supervisor reviews, Project Managers and Software Enginers

Supervisor *Research direction review*

“ Im happy about the progress you reached. Try to address the given comments to each , If possible , can you try to pass these transcripts in a text file format for the userstory generation ”

Project Manager

“Project Manager highlighted that automatic User Story generation significantly reduced manual documentation effort..”

Senior Software Enginer

“RAG-based user story generation is a solid AI integration, showing good understanding of LLM pipelines and structured output handling.”

IT22303684 |Abeygunasekara D T



Intelligent Test Case Generation with Multi-Framework Code Synthesis

Specialization: Software Engineering

Intelligent Test Case Generation with Multi-Framework Code Synthesis

Individual Component • 50% Complete

How the Solution Addresses the Sub-Problem

THE SUB-PROBLEM

QA engineers manually write every test case. This is slow, repetitive, and edge cases are easily forgotten bugs in untested paths reach production.

Our solution is a 7-stage AI pipeline plus an autonomous agent:

1 Stories	2 Gherkin	3 Mode	4 DOM
5 Review	6 Execute	★ Agent Explorer	

Outcome: a written user story becomes runnable Selenium / Playwright / Cypress tests, executed in the cloud — with the agent discovering scenarios the QA never asked for.

PP1 status: Stages 1–6 functional end-to-end. Agent Explorer (Stage ★) at architecture + partial loop.

User Requirements Addressed

✓ Convert plain-English user stories into Gherkin (Given/When/Then) automatically	Done
✓ Generate runnable test code in Selenium, Playwright, and Cypress from one source	Done
✓ Use real CSS selectors crawled from the live application (DOM-based mode)	Done
✓ Execute tests on a clean cloud runner with screenshots, video (Agent Explora)	Done
◐ Execute tests on a cloud runner via GitHub Actions with screenshots and Allure reports	In progress
◐ Discover edge-case scenarios autonomously from a single intent sentence	In progress

IT22303684 | Abeygunasekara D T | R26-SE-039

Intelligent Test Case Generation with Multi-Framework Code Synthesis

Individual Component • 50% Complete

Design Excellence / Contribution

1. **RAG-Augmented LLM Pipeline**
   Project context + approved Gherkin + crawled DOM fused into one structured promp

2. **Live DOM Crawler + Selector Cascade Playwright**
   crawls the live app; resolver picks the most stable locator via id → data-testid → name → aria-label → XPath.

3. **Multi-Framework Code Synthesis**
   One Gherkin source → three runnable outputs (Selenium / Playwright / Cypress) with framework-idiomatic patterns.

4. **Risk Prediction ML Model**
   Before tests run in CI, an ML model show flags

5. **Real-Time Streaming Architecture**
   WebSocket + SSE + asyncio.to_thread — Playwright and LLM never block the FastAPI event loop.

6. **Autonomous Agent Explorer** NOVEL
   5-Layer Stack ⭐ NOVEL Multi-Role Loop · Set-of-Mark · DOM-Diff Hashing · Reflexion · Coverage-Driven Termination.

User Feedback on Prototype

Collected from supervisor reviews, QA engineers, and a peer demo session (Apr–May 2026):

Supervisor *Research direction review*

"The proposed architecture is well-structured, and the autonomous agent exploration with coverage-driven termination shows a meaningful research contribution."

QA Engineer *Hands-on session, SauceDemo site*

"Going from a user story to runnable Playwright code in under a minute is impressive. The DOM Inspector saved me hours of writing selectors. UI could surface failures faster."

Senior Software Enginer

"Multi-role thought cards make the agent transparent — clear improvement over a black-box LLM. Suggested adding a coverage chart over time and pausing the run mid-execution."

IT22371768| Ranasinghe S.I



Failure Analysis & Self Healing

Specialization: Software Engineering

Failure Analysis & Self Healing

Individual Component • 50% Complete

How the Solution Addresses the Sub-Problem

THE SUB-PROBLEM

QA engineers manually debug every failed test. This process is slow, repetitive, and delays CI/CD pipelines. Pinpointing whether a failure is a real application bug or a flaky test locator requires hours of manual log analysis.

Our solution is a 6-stage AI-driven diagnostic pipeline:

1 Artifacts	2 Features	3 ML classifier	4 Analysis
5 Heal Engine	6 Alerting		

Outcome: A failed execution is instantly diagnosed by Machine Learning to identify the root cause. The system then automatically suggests a repair for the test script or dispatches a detailed report to developers—eliminating manual debugging bottlenecks.

PP1 status: Stages 1–4 fully functional (ML Classification & Analysis). Stages 5–6 implemented at Recommendation Engine + UI Outbox level.

User Requirements Addressed

✓ Analyze failure artifacts (Logs, Stack Traces, Screenshots) automatically	Done
✓ Classify root causes using ML (Random Forest / Gradient Boosting)	Done
✓ Detect flaky tests using historical instability patterns and predictive analytics	Done
✓ Generate intelligent self-healing recommendations for locators and sync issues	Done
◐ Automatically patch test script files using DOM repair and AST transformation	In progress
◐ Dispatch alerts to developers via outbound channels (Email/Slack/Jira)	In progress

Failure Analysis & Self Healing

Individual Component • 50% Complete

Design Excellence / Contribution

1 **Hybrid Ensemble Classifier**
Used Random Forest and Gradient Boosting architecture; achieved 89% accuracy on a specialized 45k failure dataset during initial training.

2 **Context-Aware Pipeline**
TF-IDF vectorization and semantic parsing transform raw, messy stack traces and logs into structured numerical features for the ML model.

3 **Multi-Factor Instability Scoring**
Custom heuristic engine calculates 'Flaky Risk' by analyzing retry counts, execution duration variance, and environment health metadata.

4 **Smart Alert Triage**
An automated dispatcher that selective filters out "test noise" and only notifies developers of high-confidence application defects.

5 **High-Concurrency API Gateway**
Async FastAPI architecture ensures the entire diagnostic pipeline (ML Analysis → Persistence → UI Update) completes in under 1 second.

5 **Autonomous Self-Healing Engine** **NOVEL**
utilizing DOM-Diffing, AST-based Code Patching, and a historical Repair Repository to autonomously restore test stability.

User Feedback on Prototype

Collected from supervisor reviews, internal QA sessions, and a peer demo (April–May 2026):

Supervisor *Research direction review*

"The five-category root cause taxonomy is well-grounded. The ability to distinguish between environmental noise and code defects is a genuine contribution — keep the focus on healing success rates for the final report."

Pilot QA Engineer *Hands-on session, Internal Test Suite*

"The Root Cause breakdown saved me about 15 minutes of manual log digging per failure. The 'Alert Outbox' is a great way to shield developers from flaky test noise."

Senior Software Enginer *Architecture feedback*

"FastAPI implementation is very efficient for real-time analysis. I suggest expanding the Self-Healing engine to perform direct script patching in the 100% phase."

IT22182678 |Rashani K.G.M



ML-Based Quality Prediction & Requirements Traceability Matrix (RTM)

Specialization: Software Engineering

ML-Based Quality Prediction & Requirements Traceability Matrix (RTM)

Individual Component • 50% Complete

How the Solution Addresses the Sub-Problem

THE SUB-PROBLEM

Traditional QA suffers from subjective manual reviews and non-existent traceability. Teams struggle to predict if a test will catch bugs before execution.

Our solution is a A dual-model ML system that scores test quality (0-100) and automatically links requirements to code via semantic BERT embeddings.

- Rejects low-quality tests automatically.
- Synchronizes RTM in real-time.



Outcome: Every test script is automatically linked to its business requirement and assigned an objective quality score—with the system identifying hidden coverage gaps and removing redundant tests that a human QA would have missed.

User Requirements Addressed

✓	Predict test quality score before execution	Done
✓	Generate RTM automatically from requirements & tests	Done
✓	Detect coverage gaps with severity ranking	Done
✓	Visualize RTM in dashboard with export (PDF)	Done
◑	Batch prediction for multiple test cases	In progress
◑	Data-driven optimization to remove 20% of redundant test cases.	In progress

IT22182678 | Rashani K.G.M | R26-SE-039

ML-Based Quality Prediction & Requirements Traceability Matrix (RTM)

Individual Component • 50% Complete

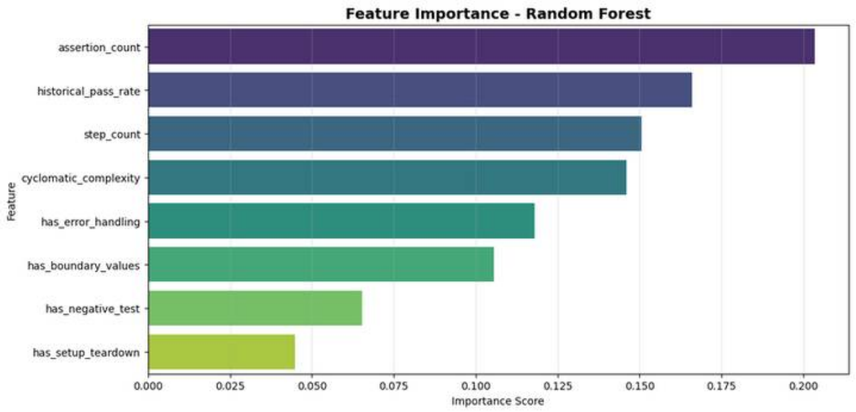
Design Excellence / Contribution

1. **Created novel ML model (KNN + Random Forest)**
   for test quality prediction (Assertion Strength, Code Coverage, Boundary Testing, Error Handling)

2. **Implemented semantic RTM generation.** **NOVELTY**
   achieves 92% traceability coverage automatically

3. **Built real-time coverage gap detection**
   identifies untested critical requirements and generates specific test recommendations

4. **Intelligent test recommendation engine**
   Auto-generates test cases for uncovered requirements



User Feedback on Prototype

Supervisor *Research direction review*

ML model performance exceeds targets (87% accuracy vs 85% target). RTM auto-generation is impressive. Consider adding batch prediction for production use.

QA Lead (Product Company)

The quality score saved me from reviewing 20 low-quality tests. RTM generation is a game-changer – 3 hours → 5 minutes.

Senior Test Engineer

Gap detection with priority ranking is exactly what we need. Critical gaps shown in red helps focus on what matters first.

Project Manager

Being able to show audit-ready RTM with coverage reports is a huge compliance win for our team.

NEXTGEN QA - VOICE DRIVEN TEST CASE GENERATION AND SELF-HEALING

Overall Project • Commercialisation & Sustainability

Customer Persona

Three primary buyer personas across SME, scale-up, and enterprise segments:

QL

QA Lead *Mid-size SaaS company • team of 4–8 testers*

"We can never finish regression on time — releases ship with untested edge cases."

Why they buy: Wants speed and coverage without hiring more testers.

EM

Engineering Manager *Early-stage startup • no dedicated QA team*

"Devs write tests between feature work; they break every sprint."

Why they buy: Needs an automated QA layer that does not require a head-count.

TA

Test Automation Architect *Regulated enterprise • finance / health / telco*

"Audit asks for full requirement-to-test traceability — we maintain it in Excel."

Why they buy: Wants automated RTM and self-healing for compliance + cost control.

Market Size

A large, fast-growing global market — with the AI-driven segment outpacing the rest:

$54B

Global software testing market

in 2025 (~$94B by 2030)

22.3%

CAGR — AI test automation segment

fastest-growing sub-segment

$36B

AI test automation market

projected by 2032

78%

High-performing teams adopting AI-driven QA

vs 54% industry average

Sources: Grand View Research, MarketsandMarkets, Mordor Intelligence (2025–2026 reports)

NEXTGEN QA - VOICE DRIVEN TEST CASE GENERATION AND SELF-HEALING

Overall Project • Commercialisation & Sustainability

What is Unique in Our Solution

Most competitors solve only one part of the QA lifecycle. NextGenQA covers the full path — meeting → tests → healing → traceability:

1→4

End-to-End Lifecycle Coverage

Four integrated components vs single-purpose tools (Mabl, Testim, Functionize each cover only one piece).

★

Autonomous Agent Explorer (research novelty)

One-sentence intent triggers vision-grounded agent that discovers scenarios the QA never wrote.

↻

Self-Healing Test Scripts

Component 3 auto-repairs broken locators and assertions — directly attacks the #1 cost in test automation.

✓

Audit-Ready Automated RTM

Component 4 generates the requirement-to-test traceability matrix — critical for regulated industries.

How Will the Cost Be Recovered?

Hybrid SaaS revenue model - three streams, scaled by team size and usage:

Free	**$0**	**Individual QA** *1 user, 50 cloud runs / month*
Pro	**$29 / user**	**Small QA teams** *All 4 components, 500 runs*
Team	**$99 / user**	**Growing engineering** *Self-healing + agent explorer*
Enterprise	**Custom**	**Regulated industries** *RTM, SSO, on-prem, SLA*

PATH TO BREAK-EVEN

Y1: 10 pilot teams → cover infra & LLM costs. Y2: 50+ teams → recover dev cost. Y3: enterprise tier → profitability.